Matemática para Aprendizaje Automático

Clase 4.2 — Eficiencia Computacional

Pablo A. Iturralde · Universidad Católica del Uruguay · 2026

Plan de la clase

Factores de eficiencia: qué determina la velocidad de un cálculo

Conteo de operaciones: cuántas cuentas hace un algoritmo

Notación Big O: cómo describimos el escalamiento asintótico

Vectorización y broadcasting: programación eficiente en Python

Paralelización y software: cómo se aprovecha el hardware moderno

Disclaimer: las diapositivas de clase son un breve resumen de los temas tratados. Las mismas no sustituyen la lectura de los textos recomendados.

¿Por qué importa la eficiencia?

El costo de computar

Entrenar modelos de ML a gran escala tiene un costo real:

- **Microsoft** firmó un contrato a 20 años para reactivar la central nuclear Three Mile Island (835 MW) y alimentar sus data centers
- **Google** compró la empresa de energía renovable Intersect Power por **$4.750M** y planea invertir $185.000M en infraestructura de data centers en 2026
- **TSMC** tiene su producción de chips vendida hasta fines de 2027
- Entrenar GPT-4 costó entre **$78M y $100M** solo en cómputo. Los modelos de próxima generación podrían superar los $1.000M

La eficiencia determina si un proyecto es viable o no.

Eficiencia en todas las escalas

No es solo un problema de *Big Tech*:

Escala	Ejemplo	¿Por qué importa?
Personal	Notebook, teléfono	Memoria y batería limitadas
Empresa	Servidor, cluster	Costo de cloud, latencia
Gran escala	Data center	Energía, refrigeración, viabilidad

En todos los casos: **más eficiencia = menos hardware, menos energía, menos tiempo, menos $$$.**

¿Qué determina la eficiencia?

Cuatro factores

El tiempo que tarda un cálculo depende de:

1. **Algoritmo utilizado** — cuántas operaciones realiza
2. **Hardware subyacente** — CPU vs. GPU, velocidad, número de cores
3. **Stack de software** — bibliotecas, compiladores, optimizaciones
4. **Programación eficiente** — vectorización, broadcasting, paralelización

Hoy nos enfocamos en **1** y **4**: lo que pueden controlar como programadores y como diseñadores de algoritmos.

Parte 1

Conteo de operaciones

¿Cuántas cuentas hace un algoritmo?

Para comparar algoritmos, contamos las **operaciones aritméticas** que realizan en función del tamaño del problema.

Un **flop** (floating point operation) es una operación aritmética elemental: $+$, $-$, $\times$, $\div$.

Ejemplo: la suma de los primeros n enteros.

```python
# Algoritmo 1: n sumas
def suma_lineal(n):
    resultado = 0
    for i in range(1, n+1):
        resultado += i
    return resultado

# Algoritmo 2: 1 suma, 1 producto, 1 división
def suma_directa(n):
    return n * (n + 1) // 2
```

Algoritmo 1: n operaciones. Algoritmo 2: 3 operaciones.

Operaciones vectoriales

Para vectores $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$:

Operación	Multiplicaciones	Sumas	Total
Producto punto $\mathbf{x}^\top \mathbf{y}$	n	$n-1$	$2n-1$
Norma $\lvert\mathbf{x}\rvert$	n	$n-1$	$2n-1+1$ (raíz)
Escalar por vector $\alpha\mathbf{x}$	n	0	n
Suma $\mathbf{x}+\mathbf{y}$	0	n	n

Todas son $\sim n$ operaciones. El costo crece linealmente con la dimensión.

Producto matriz-vector

Para $A \in \mathbb{R}^{n \times n}$, $\mathbf{x} \in \mathbb{R}^n$, cada componente de $\mathbf{c} = A\mathbf{x}$ es:

$$c_i = \sum_{j=1}^{n} a_{ij} x_j$$

Cada c_i requiere n multiplicaciones y $n - 1$ sumas.

Hay n componentes, así que el total es:

$$n \cdot (2n - 1) = 2n^2 - n \approx 2n^2 \text{ flops}$$

Producto matricial: la estructura

Para $C = AB$ con $A, B \in \mathbb{R}^{n \times n}$, cada elemento de C es:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

```
# Triple loop
for i in range(n):          # fila de A
    for j in range(n):      # columna de B
        for k in range(n):  # suma
            C[i][j] += A[i][k] * B[k][j]
```

Tres índices anidados, cada uno recorriendo n valores.

Producto matricial: el conteo

Cada c_{ij} requiere n multiplicaciones y $n-1$ sumas.

Hay n^2 elementos en C.

Total:

- Multiplicaciones: $n^2 \cdot n = n^3$
- Sumas: $n^2 \cdot (n-1) = n^3 - n^2$
- **Total: $2n^3 - n^2$ flops**

Para $n = 1000$: $\approx 2 \times 10^9$ operaciones (2 mil millones).

Tabla de costos (hasta ahora)

Operación	Flops exactos	Para $n = 1000$
Producto punto $\mathbf{x}^\top \mathbf{y}$	$2n - 1$	$\sim 2 \times 10^3$
Matriz-vector $A\mathbf{x}$	$2n^2 - n$	$\sim 2 \times 10^6$
Producto matricial AB	$2n^3 - n^2$	$\sim 2 \times 10^9$

$A\mathbf{x} = \mathbf{b}$: dos caminos al mismo resultado

Queremos resolver el sistema lineal $A\mathbf{x} = \mathbf{b}$ con $A \in \mathbb{R}^{n \times n}$ invertible.

Camino 1 — Inversión: calcular A^{-1}, luego $\mathbf{x} = A^{-1}\mathbf{b}$

Camino 2 — Eliminación gaussiana: reducir $[A|\mathbf{b}]$ a forma triangular, resolver por sustitución

Ambos dan la misma respuesta (o casi). ¿Cuestan lo mismo?

Camino 2: eliminación gaussiana

Eliminación hacia adelante (forward elimination): convertir A en triangular superior.

En el paso k: eliminar el elemento (i, k) para todas las filas $i > k$. Esto involucra actualizar $\sim (n-k)$ filas, cada una con $\sim (n-k)$ operaciones.

Costo total de la eliminación: $\sum_{k=1}^{n-1} 2(n-k)^2 \approx \dfrac{2n^3}{3}$ flops

Sustitución hacia atrás (back substitution): $\sim n^2$ flops

Total: $\sim \dfrac{2n^3}{3} + n^2$ flops

Camino 1: inversión matricial

Invertir A equivale a resolver n sistemas: $A\mathbf{e}_i = \mathbf{x}_i$ para cada columna $\mathbf{e}_i$ de la identidad.

Costo de A^{-1}: $\sim 2n^3$ flops

Costo de $A^{-1}\mathbf{b}$: $\sim 2n^2$ flops (un producto matriz-vector)

Total: $\sim 2n^3 + 2n^2$ flops

La inversión es aproximadamente 3× más costosa que la eliminación gaussiana. (y menos precisa también)

Resumen: tabla de costos

Operación	Flops
Producto punto $\mathbf{x}^\top \mathbf{y}$	$2n $
Matriz-vector $A\mathbf{x}$	$2n^2$
Producto matricial AB	$2n^3$
Eliminación gaussiana ($A\mathbf{x} = \mathbf{b}$)	$\frac{2}{3}n^3$
Inversión matricial (A^{-1})	$\sim 2n^3$

Notación Big O

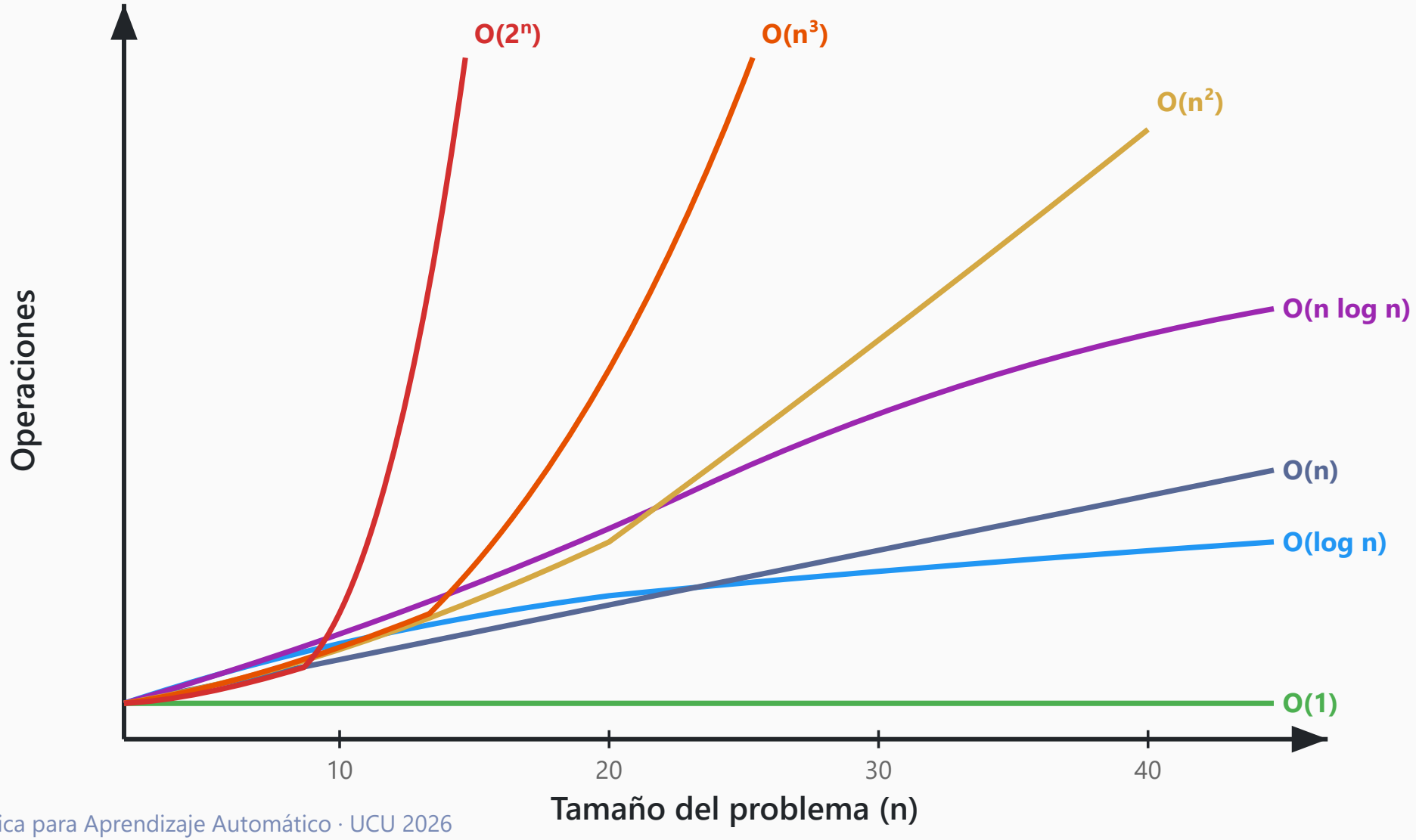
Una herramienta para describir escalamiento

Definición: decimos que $f(n) = O(g(n))$ si existen constantes $C > 0$ y n_0 tales que:

$$f(n) \leq C \cdot g(n) \quad \text{para todo } n \geq n_0$$

Idea: $g(n)$ es una **cota superior** de la tasa de crecimiento de $f(n)$.

Clases de complejidad



Big O descarta "detalles"

Big O retiene el **término dominante** y descarta constantes y términos menores.

$$2n^3 - n^2 \quad \longrightarrow \quad O(n^3)$$

$$\frac{2}{3}n^3 + n^2 \quad \longrightarrow \quad O(n^3)$$

Ambos son $O(n^3)$. Pero el primero cuesta **3× más** que el segundo.

Big O nos dice que ambos **escalan igual**, pero no nos dice cuál es más rápido.

La tabla con Big O

Operación	Flops exactos	Big O
Producto punto $\mathbf{x}^\top \mathbf{y}$	$2n - 1$	$O(n)$
Matriz-vector $A\mathbf{x}$	$2n^2 - n$	$O(n^2)$
Producto matricial AB	$2n^3 - n^2$	$O(n^3)$
Eliminación gaussiana	$\frac{2}{3}n^3 + n^2$	$O(n^3)$
Inversión matricial	$\sim 2n^3$	$O(n^3)$

Ambas herramientas son útiles para preguntas diferentes.

Limitaciones de Big O

Las constantes importan. Un algoritmo $5n^2$ le gana a uno $100n^2$, aunque ambos son $O(n^2)$.

Para n chico, los términos menores pueden dominar. n^3 vs. $10n^2$ para $n < 10$

Ejemplo: Strassen. La multiplicación matricial de Strassen es $O(n^{2.81})$ vs. $O(n^3)$ del algoritmo clásico. Pero tiene constantes más grandes, y solo conviene para matrices de más de $\sim 100 \times 100$.

Parte 2

Implementación eficiente de operaciones algebraicas

Vectorización

Ejemplo: producto punto

```python
# Con loop — lento
def dot_loop(x, y):
    resultado = 0
    for i in range(len(x)):
        resultado += x[i] * y[i]
    return resultado

# Vectorizado — rápido
import numpy as np
resultado = np.dot(x, y)
```

Para $n = 10^6$: el loop tarda ~500 ms, `np.dot` tarda ~1 ms.

Mismo conteo de operaciones. ~500× diferencia en tiempo.

Idéntica cantidad de operaciones, pero la velocidad difiere por **100–1000×**.

¿Por qué?

Vectorización

Loop:

- **Overhead de interpretación:** cada iteración del loop pasa por el intérprete de Python
- **Llamadas a función:** cada operación aritmética es una llamada de Python
- **Patrones de memoria:** el loop no optimiza el acceso a caché

Versión vectorizada:

- **Código pre-compilado:** las operaciones se ejecutan con código optimizado en lenguaje de bajo nivel (C/Fortran)
- **Uso eficiente del hardware** las implementaciones de las bibliotecas usan el hardware de forma eficiente, por ejemplo instrucciones *SIMD (Single Instruction Multiple Data)*

Conclusiones

En general:

- si una función tiene una implementación provista por una biblioteca optimizada (NumPy, SciPy, etc.), es mejor usarla que escribir un loop manual.
- los loops en lenguajes de alto nivel deben evitarse en lo posible.

Broadcasting

Ejemplo: centrar datos

Un caso típico en ML: restar la media por columna a una matriz de datos.

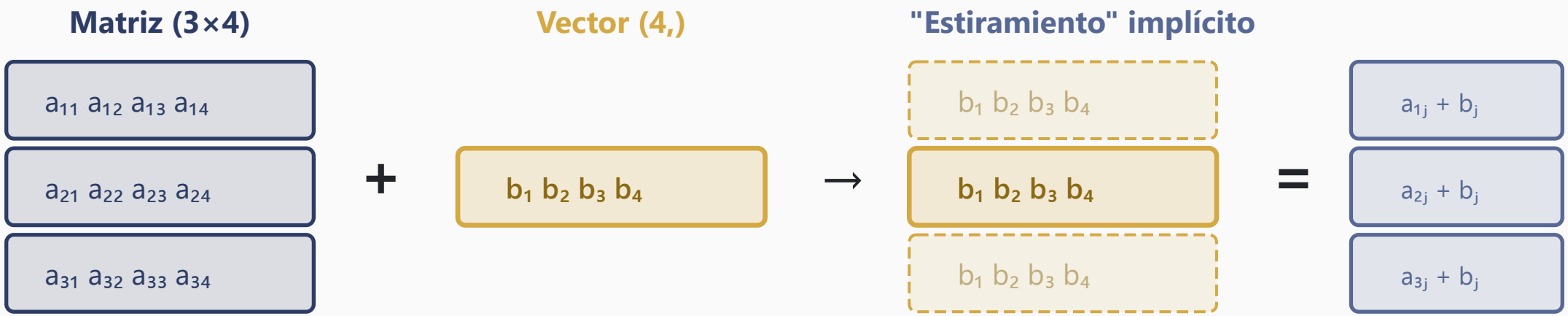
```
# X = matriz de datos (n, d) a centrar
# Con loop
medias = np.mean(X, axis=0)          # vector (d,)
for i in range(n):
    #Restamos la media a cada columna
    X_centrado[i, :] = X[i, :] - medias
```

```
# Con broadcasting — rápido y elegante
X_centrado = X - np.mean(X, axis=0)  # (n, d) - (d,)
```

NumPy "estira" el vector de medias $(d,)$ a (n, d) automáticamente.

Operaciones entre arrays de distinta forma

Broadcasting permite operar arrays de formas diferentes sin copiar datos.



NumPy no copia el vector — opera como si estuviera replicado

```
# Con loop: O(n·d) y lento
for i in range(n): X[i,:] -= media
# Con broadcasting: X - media ← una línea
```

NumPy "estira" implícitamente el array más chico para que las formas coincidan. No copia los datos en memoria n veces.

Paralelización

Dividir el trabajo

Algunas operaciones pueden dividirse en sub-tareas **independientes** que se ejecutan en paralelo, aprovechando las arquitecturas modernas *multi-core* en CPUs y GPUs.

Producto matricial como ejemplo:

$$C = AB \quad \Longrightarrow \quad C_{ij} = \sum_k A_{ik} B_{kj}$$

Cada elemento C_{ij} se puede calcular de forma independiente. Podemos repartir filas (o bloques) de C entre distintos procesadores.

Esto es lo que hace que las GPUs (con miles de cores) sean tan eficientes para álgebra lineal.

¿Qué se puede paralelizar?

No todo es paralelizable. La condición es que las sub-tareas sean **independientes**.

Operación	¿Paralelizable?	¿Por qué?
Producto matricial AB	Sí	Cada C_{ij} es independiente
Matriz-vector $A\mathbf{x}$	Sí	Cada c_i es independiente
Sustitución hacia atrás	No	x_i depende de $x_{i+1}, \ldots, x_n$
Eliminación gaussiana	Parcial	Cada paso k depende del anterior

La dependencia secuencial en algoritmos impide paralelizarlos.

Paralelizabilidad de algoritmos de aprendizaje = permite entrenar modelos gigantes en hardware moderno.

Parte 3

Otros aspectos que hacen a la eficiencia

Software stack

Cuando escribimos `np.linalg.solve(A, b)`, esto es lo que pasa:

```
Python / NumPy          ← interfaz de alto nivel
       ↓
    LAPACK              ← algoritmos (factorizaciones, solvers)
       ↓
     BLAS               ← operaciones básicas optimizadas
  (OpenBLAS, IntelMKL,        para el hardware específico
   cuBLAS, ...)
       ↓
   Hardware             ← CPU, GPU, TPU
```

Cada capa optimiza lo que le corresponde.

Tanto LAPACK como BLAS tienen implementaciones optimizadas instrucción por instrucción para cada arquitectura.

Explotar la estructura del problema

En ciertos contextos surgen problemas con estructura especial. Por ejemplo:

- Cálculo de valores propios en una matriz simétrica
- Resolución de sistemas $Ax = b$ con matrices triangulares
- Solución de sistemas con matrices dispersas (sparse: pocos valores no nulos)

Las bibliotecas de álgebra lineal tienen *solvers* especializados que explotan la información conocida sobre la estructura del problema (e.g. `eig` vs `eigh` para valores propios, aprovecha la simetría).

En general, utilizarlos reduce la complejidad computacional y aumenta la precisión numérica.

Resumen

El costo del cómputo depende de:

1. **Elegir los algoritmos apropiados** para la tarea.
2. **Implementar de forma eficiente** usando, por ejemplo, vectorización, broadcasting y paralelización.
3. **Usar bibliotecas optimizadas** y solvers especializados (no reinventar la rueda).

¿Preguntas?